Лабораторная работа №1

Пшембаева Р.Н. гр.4404

Задача 1. Написать программу где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками.

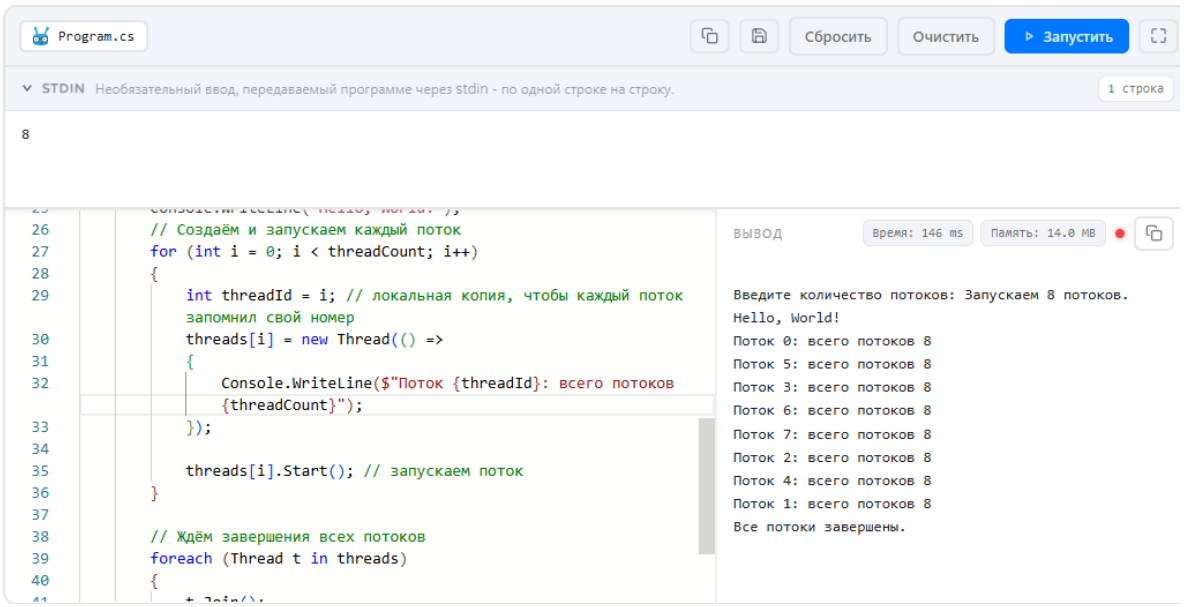



Рис. 1. Вывод при количестве потоков 8

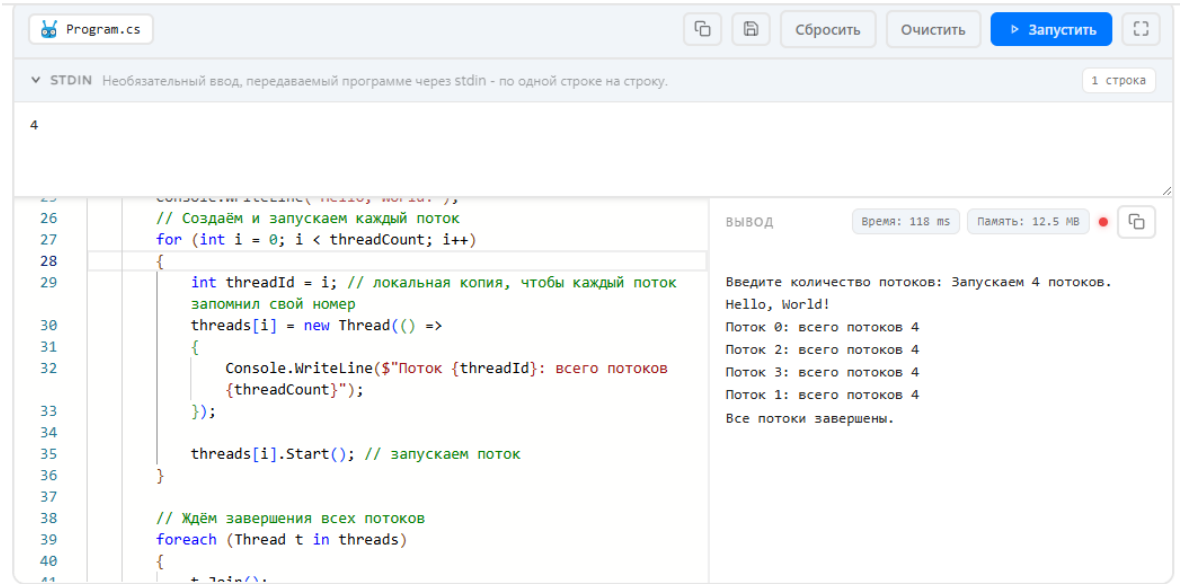



Рис. 2. Вывод при количестве потоков 4

Порядок вывода идентификаторов недетерминирован, потому что потоки выполняются параллельно и не синхронизированы. Планировщик операционной системы решает, какой поток когда получит процессор, и это решение зависит от загрузки системы, числа ядер и других факторов.

Задача 2. Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива:

b[i] = (a[i-1] + a[i] + a[i+1])/3.0

Запустите программу с 8-ю процессами при различных типах распределения работ.

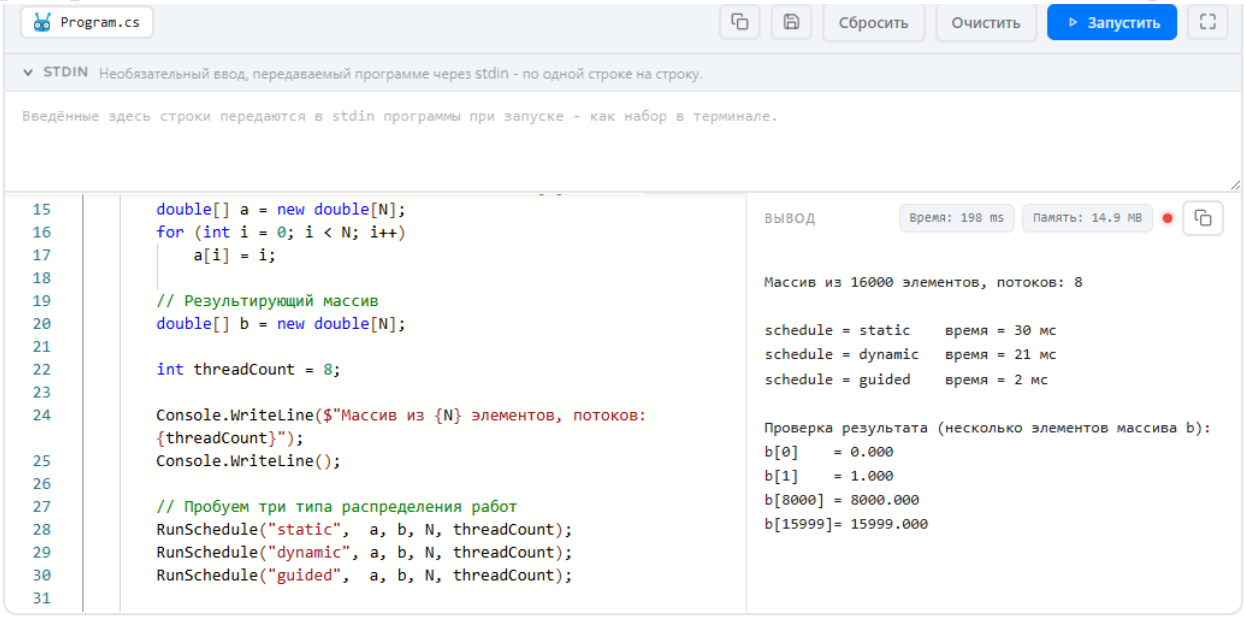



Рис. 3.

Задача3. Модифицируйте задачу 1 так, что бы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения. Постарайтесь найти как можно больше.

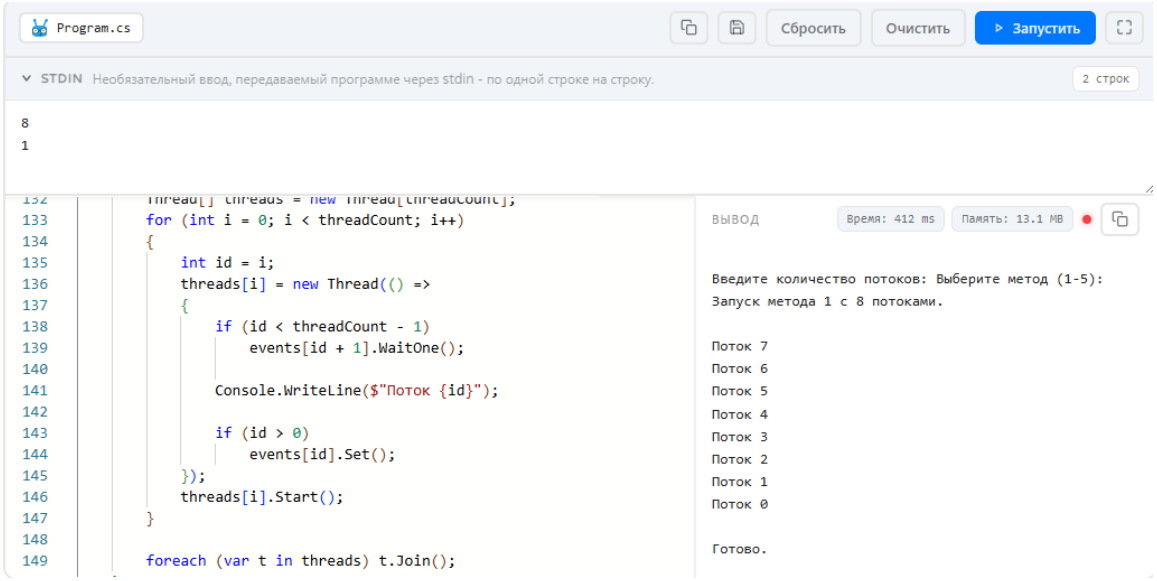



Рис.4. Метод 1 при количестве потоков 8

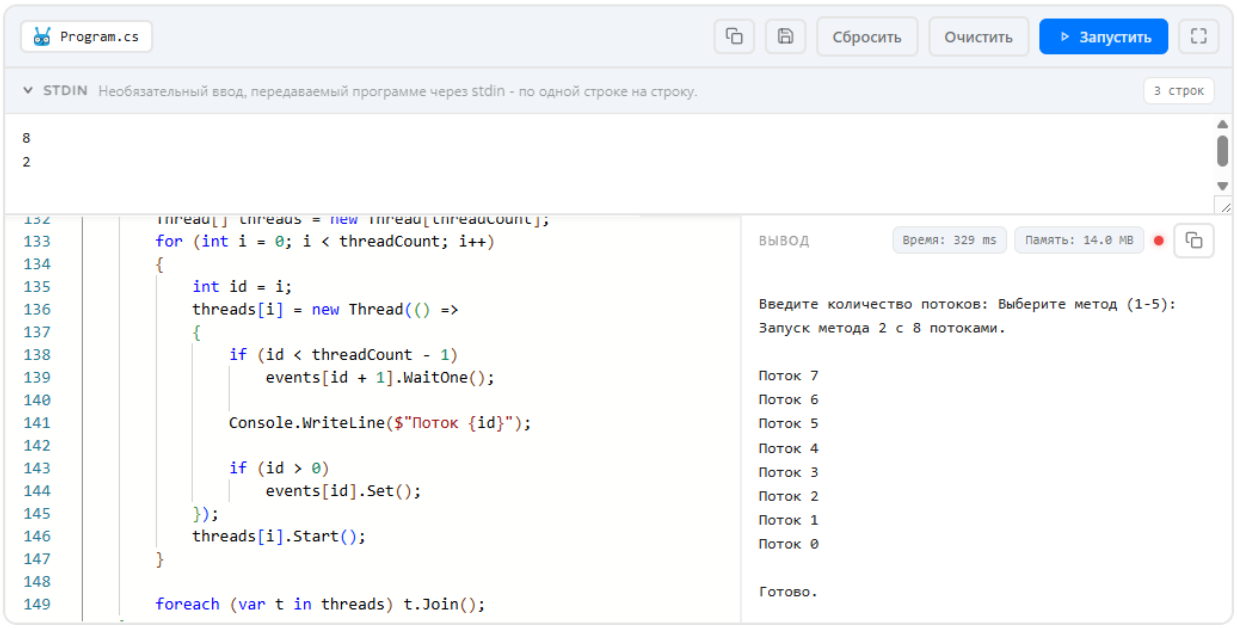



Рис.5. Метод 2 при количестве потоков 8

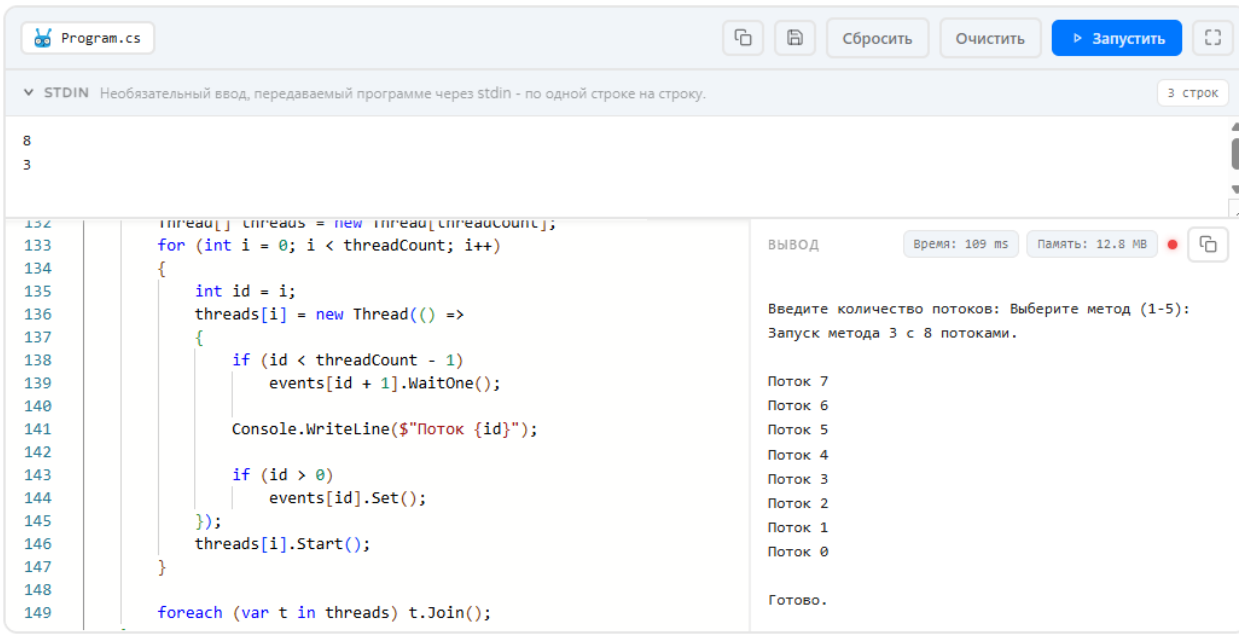



Рис.6. Метод 3 при количестве потоков 8

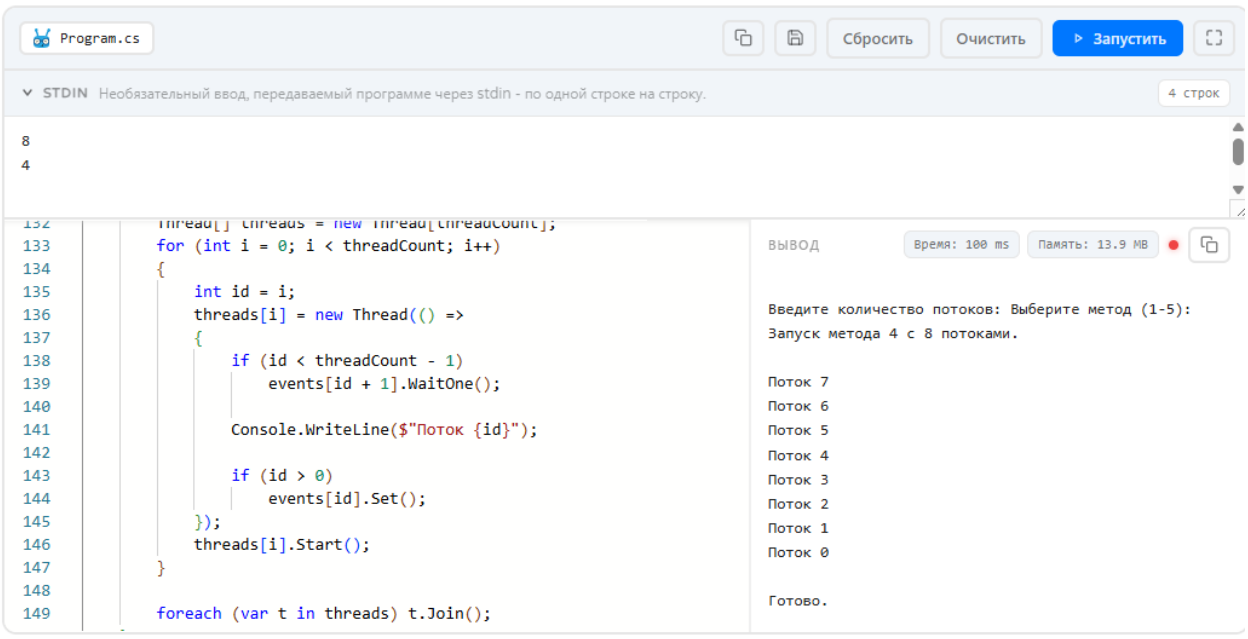



Рис.7. Метод 4 при количестве потоков 8

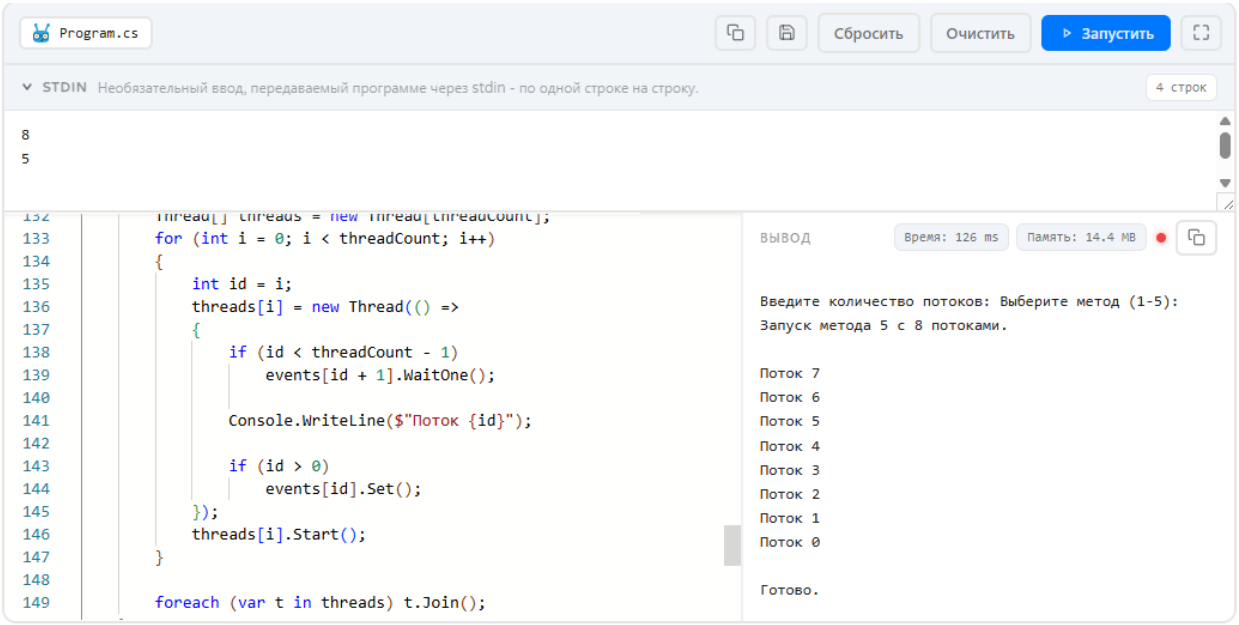



Рис.8. Метод 5 при количестве потоков 8

**Вывод**: в ходе выполнения лабораторной работы были изучены базовые принципы параллельного программирования и реализованы три задачи с использованием многопоточности на языке C# (как альтернатива OpenMP).

**В первой задаче** была реализована программа, в которой каждый поток выводит свой идентификатор. В результате выполнения было установлено, что при отсутствии явной синхронизации порядок вывода идентификаторов является **недетерминированным** (случайным). Это связано с тем, что операционная система (ее планировщик) самостоятельно распределяет процессорное время между потоками, и предугадать, какой поток выполнится первым, невозможно. При этом состав выводимых данных всегда остается одинаковым (каждый идентификатор появляется ровно один раз).

**Во второй задаче** было реализовано параллельное вычисление средних значений элементов массива (16000 элементов). Были изучены и протестированы три типа распределения работ (schedule):

static — итерации распределяются поровну заранее. Показал высокую скорость, так как не требует дополнительных затрат на синхронизацию потоков.

dynamic — каждый поток берет по одному элементу динамически. Оказался самым медленным из-за накладных расходов на атомарные операции (Interlocked) при захвате каждого индекса.

guided — элементы выдаются блоками, размер которых постепенно уменьшается. Показал компромиссную производительность: меньше накладных расходов на синхронизацию, чем в dynamic, и лучше балансировка, чем в static.

**В третьей задаче** было продемонстрировано управление порядком выполнения потоков. С помощью средств синхронизации (семафоров) удалось добиться строго обратного порядка вывода идентификаторов (от N-1 до 0). Это доказывает, что хотя потоки по своей природе асинхронны, использование примитивов синхронизации позволяет сделать поведение программы строго детерминированным.